

Manufacturing-Impact-Report

FoundryMind AI: Manufacturing Impact Report

1. Executive Summary

FoundryMind AI brings production-grade machine learning to metal casting and factory operations. It reduces scrap, rework, and downtime by combining optimization models, predictive analytics, and a process-aware LLM copilot that translates model outputs into clear operator guidance. The platform is cloud-ready (Render + Vercel) yet lean enough for on-prem deployments where connectivity is constrained.

2. Problem & Opportunity

- Foundries and discrete manufacturers face variable quality, tight margins, and skill gaps on the shop floor.
- Process engineers rely on tribal knowledge; parameter tuning is slow and inconsistent.
- Traditional MES/SCADA data is underused for forward-looking decisions.
- Opportunity: codify best practices with ML, surface high-quality setpoints, and give operators actionable guidance before defects occur.

3. Solution Overview

- Casting optimization service that evaluates melt chemistry and process settings, then proposes improved parameters to cut scrap and porosity risk.
- Factory performance prediction that estimates yield, cycle time, and defect probability for planned runs or schedule changes.
- LLM assistant that explains recommendations in process language, shortening troubleshooting loops and onboarding time.
- Web UI (Next.js) for engineers and operators; FastAPI backend for ML inference; reproducible training pipelines in `src/`.

4. Core Capabilities

- **Casting optimization:** `POST /api/casting/optimize` — recommend melt, chemistry, and setting changes before a heat hits the line, cutting scrap and porosity risk.
- **Factory forecasting:** `POST /api/factory/predict` — score upcoming runs for risk, throughput, and expected quality to protect OEE.
- **LLM explanations:** `POST /api/assistant/explain` — turn raw model outputs into stepwise guidance, so operators know what to adjust and why.

- **Health/readiness:** `GET /health` — readiness probe; model artifacts tracked in `models/` for traceability.

5. Architecture (End-to-End)

- **Frontend (Next.js/Vercel):** Operator/engineer workflows for optimization, predictions, and LLM guidance.
- **Backend (FastAPI on Render or on-prem):** Serves inference, loads artifacts, and enforces CORS and env-configured origins.
- **Models:** Casting and factory models serialized in `models/` ; trained via `src/` scripts for reproducible builds.
- **Data:** Versioned under `data/raw` and `data/processed` with synthetic data for demos and safe experimentation.
- **Orchestration:** `render.yaml` + `Dockerfile` for backend deploy; `vercel.json` + `.env` for frontend.

6. Data & Modeling

- **Sources:** Historical casting runs (chemistry, temp curves, mold data), factory telemetry, and synthetic augmentation for safe experimentation.
- **Datasets:** Factory performance data originates from a Kaggle manufacturing dataset; casting examples are synthetic to avoid exposing proprietary plant data.
- **Pipelines:** Training scripts in `src/casting/train.py` and `src/factory/train.py` ; prediction utilities in `src/factory/predict.py` and `src/casting/optimizer.py` .
- **Objectives:** Minimize scrap/rework, improve OEE, and tighten process windows while maintaining cycle time.
- **Model artifacts:** Stored in `models/` for deployment; small footprint to fit edge/on-prem constraints.

7. LLM Assistant

- Bridges model outputs and human decisions by explaining why a recommendation matters (e.g., chemistry adjustments to reduce porosity risk).
- Uses domain prompts tuned for manufacturing language, reducing ambiguity for technicians.
- Designed as a sidekick, not an autocrat: operators remain in control but get faster, clearer rationale.

8. Deployment Footprint

- **Live demo:** <https://foundry-mind-ai.vercel.app/>

- **Backend:** Deploy via Render using `render.yaml`; `Dockerfile` pins runtime and dependencies.
- **Frontend:** Deploy via Vercel with root `frontend/`; `NEXT_PUBLIC_API_BASE_URL` points to the backend.
- **Node version:** 22.x recommended; `.nvmrc / engines.node` guard against Node 24 SIGBUS issues on some Linux hosts.
- **Environment variables:** `GROQ_API_KEY`, `ALLOWED_ORIGINS`, `NEXT_PUBLIC_API_BASE_URL`.

9. Local Development (Engineer Playbook)

1. Backend (FastAPI):

- Activate your preferred Python environment (`conda ml` is typical).
- `pip install -r requirements.txt`
- (Optional) retrain: `python src/casting/train.py` and `python src/factory/train.py`
- Run: `uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000`

2. Frontend (Next.js):

- `cd frontend && cp .env.example .env.local`
- `npm install`
- `npm run dev`
- App serves on `http://localhost:3000` and calls backend via `NEXT_PUBLIC_API_BASE_URL`.

10. Factory Integration Considerations

- **Data readiness:** Start with CSV exports from MES/SCADA; evolve to streaming ingestion.
- **Change control:** Keep optimization suggestions auditable; log accepted vs. rejected recommendations.
- **Latency:** Models are lightweight; suitable for on-prem API to keep round trips under typical PLC scan times when fronted by a local service.
- **Safety:** Surface recommendations with ranges and rationale; never auto-apply without operator confirmation.

11. Security & Compliance

- HTTPS for all ingress; restrict origins via `ALLOWED_ORIGINS`.
- API keys for LLM provider stored as secrets; avoid embedding in frontend.
- Minimal PII; focus on process and equipment data.
- Containerized deployment for isolation; small attack surface.

12. Current Status

- Frontend and backend deployed; health check responds 200 on Vercel-Render path.
- Baseline casting and factory models trained and serialized in `models/`.
- Synthetic datasets available for demonstrations without exposing plant data.

13. Roadmap

- Enrich feature store with sensor fusion (temperature curves, vibration, camera-derived cues).
- Add active learning loop to retrain with operator feedback on accepted/declined recommendations.
- Add role-based access and audit logging for regulated environments.
- Expand SKU/part-specific models and automatic model selection per run.

14. How This Helps Manufacturing Teams

- **Scrap reduction:** Optimized parameters before a heat reduces porosity and inclusions.
- **Throughput:** Forward-looking risk scoring prevents schedule churn and overtime.
- **Consistency:** Standardizes best practices across shifts; accelerates onboarding for new operators.
- **Decision speed:** LLM explanations cut investigation time during deviations.

15. Contacts & Demo

- Live UI: <https://foundry-mind-ai.vercel.app/>
- Backend health: `/health`
- For a guided run with plant data, connect the backend to your MES/SCADA exports and retrain via `src/` scripts.